

```
import matplotlib.pyplot as plt
from scipy import stats

#Create the arrays that represent the values of the x and y axis
x = [5,7,8,7,2,17,2,9,4,11,12,9,6]
y = [99,86,87,88,111,86,103,87,94,78,77,85,86]

#Execute a method that returns some important key values of Linear Regression
slope, intercept, r, p, std_err = stats.linregress(x, y)

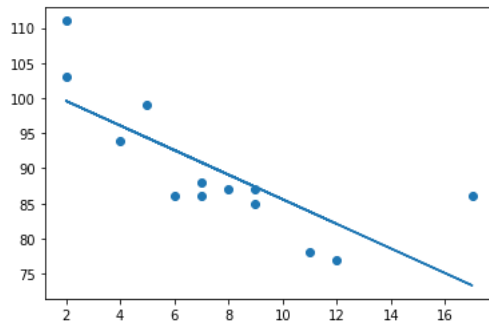
# measure the correlation
corr, _ = stats.pearsonr(x, y)
print('Pearsons correlation: %.3f' % corr)

#Create a function that uses the slope and intercept values to return a new value.
#This new value represents where on the y-axis the corresponding x value will be placed
def myfunc(x):
    return slope * x + intercept

#Run each value of the x array through the function. This will result in a new array with new values for the y-axis
mymodel = list(map(myfunc, x))

#Draw the original scatter plot & the line of linear regression
plt.scatter(x, y)
plt.plot(x, mymodel)
plt.show()
```

Pearsons correlation: -0.759



## ✓ Predict Future Values

```
from scipy import stats

x = [5,7,8,7,2,17,2,9,4,11,12,9,6]
y = [99,86,87,88,111,86,103,87,94,78,77,85,86]

slope, intercept, r, p, std_err = stats.linregress(x, y)

def myfunc(x):
    return intercept + slope * x

speed = myfunc(10)

print(speed)
```

85.59308314937454

img\_linear\_regression2.png

If x=10 then predicted y is 85.59

```
import matplotlib.pyplot as plt
from scipy import stats

#Create the arrays that represent the values of the x and y axis
x = [3,5,6,8,9,10,11,13,14,15,16,18,20]
y = [112,105,100,95,91,87,84,80,77,73,70,66,62]

#Execute a method that returns some important key values of Linear Regression
slope, intercept, r, p, std_err = stats.linregress(x, y)

# measure the correlation
```

```

corr, _ = stats.pearsonr(x, y)
print('Pearsons correlation: %.3f' % corr)

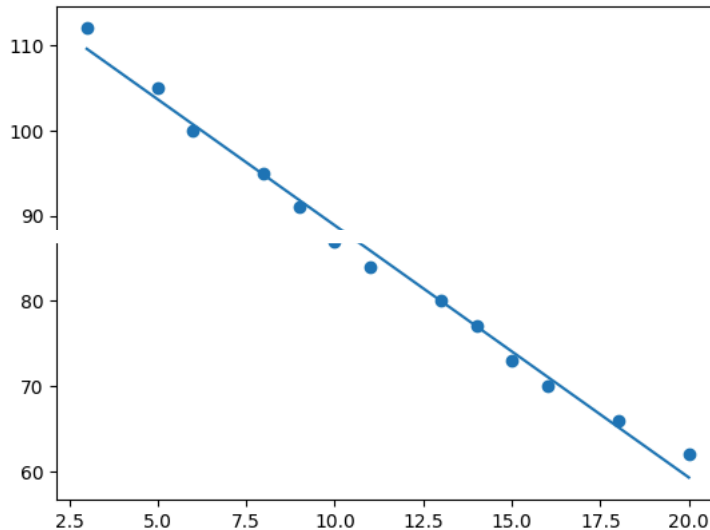
#Create a function that uses the slope and intercept values to return a new value.
#This new value represents where on the y-axis the corresponding x value will be placed
def myfunc(x):
    return slope * x + intercept

#Run each value of the x array through the function. This will result in a new array with new values for the y-a
mymodel = list(map(myfunc, x))

#Draw the original scatter plot & the line of linear regression
plt.scatter(x, y)
plt.plot(x, mymodel)
plt.show()

```

Pearsons correlation: -0.995



```

from scipy import stats

x = [3,5,6,8,9,10,11,13,14,15,16,18,20]
y = [112,105,100,95,91,87,84,80,77,73,70,66,62]

slope, intercept, r, p, std_err = stats.linregress(x, y)

def myfunc(x):
    return intercept + slope * x

speed = myfunc(10)

print(speed)

88.86966938188789

```

## Conclusion

In this exercise, I modified the original dataset and corrected a coding spelling error by replacing 'stat' with 'stats' during the SciPy regression functions. After changing the data values, I re-ran the linear regression model and observed changes in the slope, intercept, predicted value, and correlation co-efficient. The updated scatter plot still demonstrated a strong negative linear relationship, which demonstrates how linear regression models the relationship between two variables and how changing the underlying data affects the regression line and prediction results.

Start coding or [generate](#) with AI.

